American International University- Bangladesh (AIUB)
Department of Electrical and Electronic Engineering
EEE 4103: Microprocessor and Embedded Systems Laboratory

Title: **Implementation of Pedestrian Traffic Control System using an 8x8 LED Matrix using Arduino.**

Objective: The objectives of this experiment are to:

1. Introducing the fundamentals of pedestrian traffic control
2. Providing hands-on experience in programming an Arduino to interface with an 8x8 LED matrix.
3. Familiarizing with pixel manipulation techniques for dynamic visualizations.
4. Independently design and display matrix art creations based on traffic control

Theory and Methodology:

Theory:

Pedestrian Traffic Signal Systems play a crucial role in ensuring the safe and orderly movement of pedestrians across intersections. These systems often employ visual signals, such as illuminated icons and symbols, to communicate walk and don't walk commands. Utilizing an 8x8 LED matrix for such systems combines the efficiency of modern microcontroller technology with clear and versatile display capabilities.

In the context of an 8x8 LED matrix, signals are represented as a grid of LEDs, each representing a distinct part of the signal. This method is particularly effective for displaying simple yet distinct pedestrian signals, such as the walking figure or hand symbol.

Arduino, a versatile microcontroller platform, can be programmed to control the LEDs on the matrix, enabling dynamic and responsive visual displays. The theory underlying this experiment involves understanding the principles of signal creation, the concept of individual LEDs, and how these concepts translate to the physical display on an 8x8 LED matrix. It also involves mastering multiplexing, a technique used to control multiple LEDs with a limited number of pins on the microcontroller.

By integrating these principles, the Pedestrian Traffic Signal System can effectively convey crucial information to pedestrians, enhancing their safety and optimizing traffic flow.

Methodology:

The Dot Matrix LED Display Module (8x8) operates by utilizing light-emitting diodes (LEDs) arranged in an 8x8 grid to visually represent information. The working methodology involves interfacing the module with a microcontroller, typically Arduino, to control the illumination of individual LEDs. The module is divided into rows and columns, and by selectively activating specific rows and columns, the microcontroller can control the state of each LED, creating patterns and characters. The setup process entails defining the pin connections between the microcontroller and the module and specifying row and column configurations. Through programmed logic, the microcontroller can then selectively illuminate LEDs based on the desired display output. This module is commonly used in various applications, from displaying alphanumeric characters to creating dynamic visual effects in projects requiring a compact and versatile LED matrix display.

Apparatus:

1. Arduino IDE (2.0.1 or any recent version)
2. Arduino Microcontroller board
3. Dot Matrix LED Display Module (8x8) (MAX7219)
4. LEDs
5. Resistors
6. Jumper wires
7. Bread Board
8. Pushbutton

Experimental Setup:

In the Implementation of Pedestrian Traffic Control System using an 8x8 LED Matrix using Arduino, in a breadboard, 1 RED LED, 1 YELLOW LED, 1 GREEN LED is connected. The anode of the LEDs are connected to pin 3, 4 and 5 of the Arduino Uno respectively using 220Ω resistor. The cathode of the LEDs are connected to ground (GND). A pushbutton is connected to the breadboard with one of the pins connected to pin 2 of Arduino and other pin with 5v pin of Arduino. The LED matrix's VCC, GND, CLK, CS and DIN pin is connected to Arduino Uno's 5v, GND, 8, 9 and 10 pin respectively. Then the Arduino Uno is connected to the PC. From Arduino IDE, the Implementation of Pedestrian Traffic Control System using an 8x8 LED Matrix code is complied and uploaded successfully. Finally, the following output is observed –

- While the button is not pressed, the Green LED is ON and the LED Matrix is at 'Stand' state (observed in Figure 1(a) and Figure 1(b))
- When the button is pressed, the LED Matrix is at 'Go' state, the Yellow LED blinks for 5 seconds, then the Red LED is turned ON for 20 seconds and then the Yellow LED blinks again for 5 seconds and then the Green LED is ON and the LED Matrix is at 'Stand' state. (observed in Figure 1(c), 1(d) and 1(b))

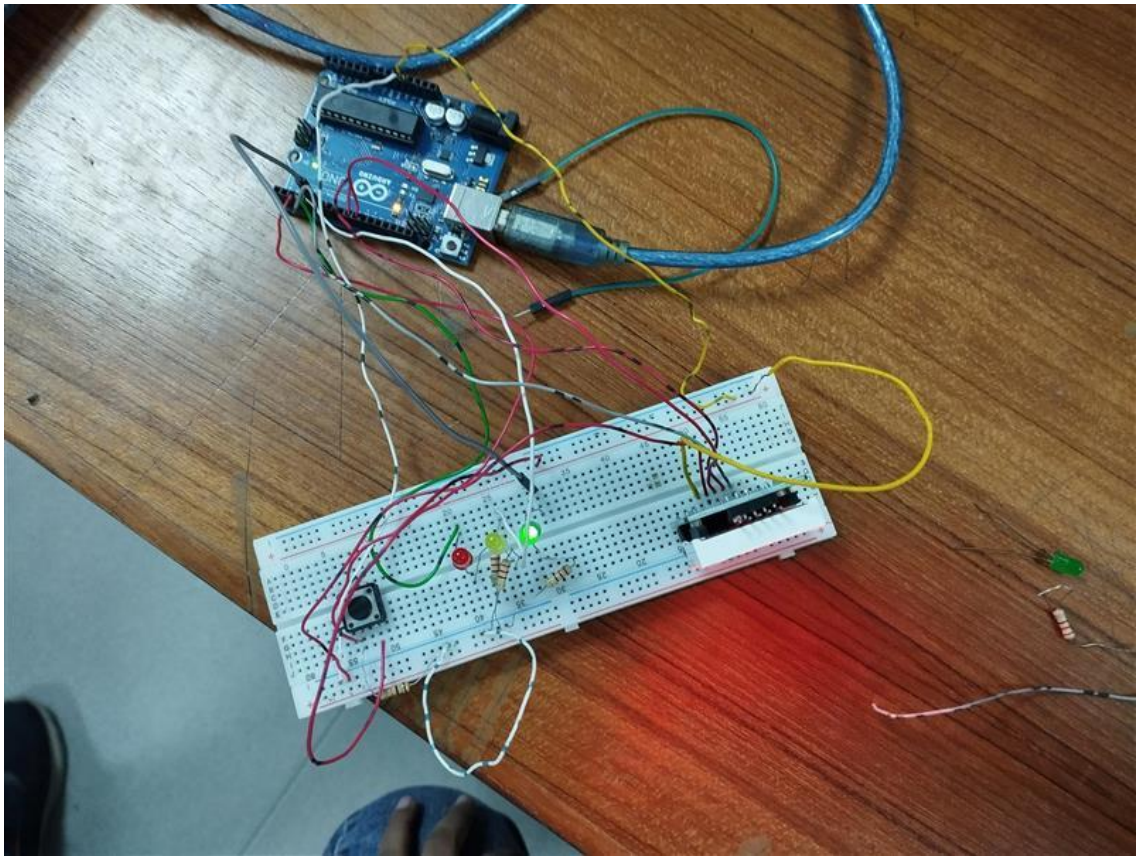

Figure 1(a): Hardware implementation of Pedestrian Traffic Control System using an 8x8 LED Matrix using Arduino (Current state: LED Matrix in 'Stand' state and Green LED ON)

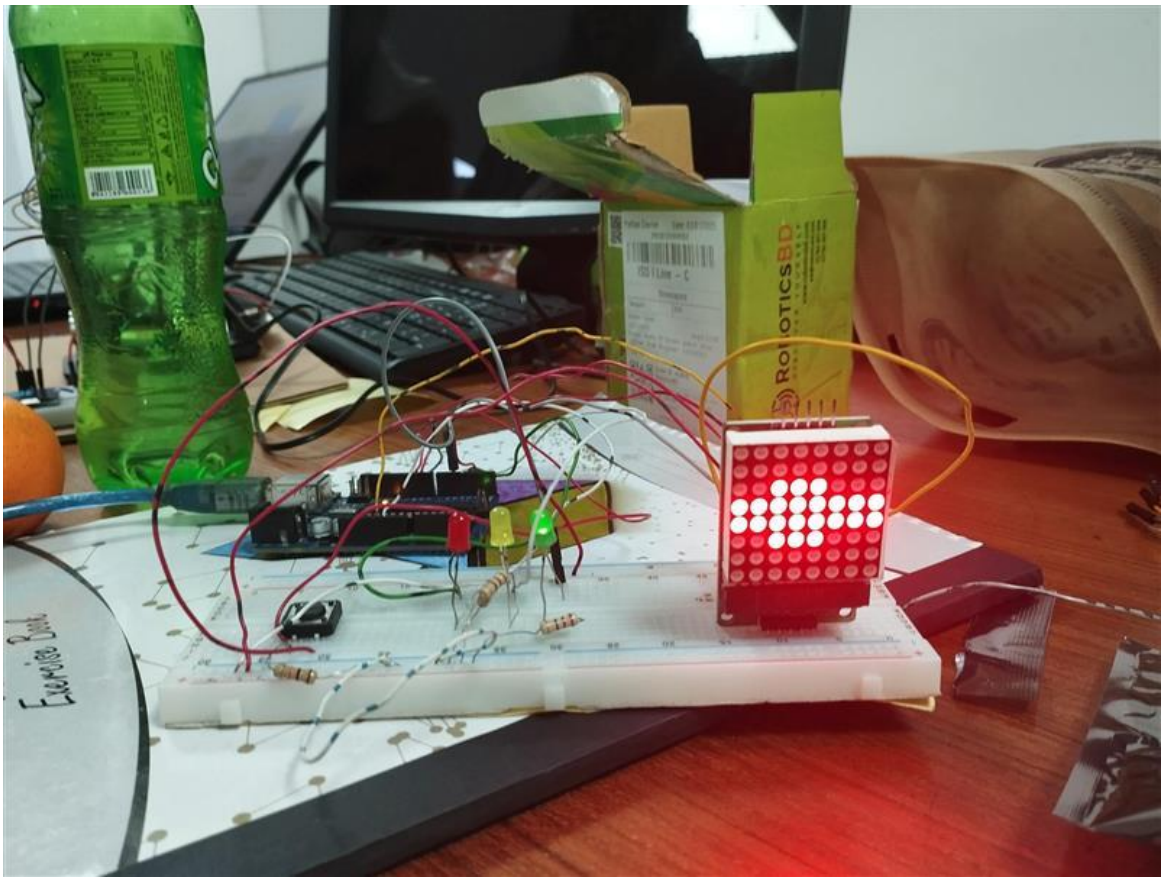

Figure 1(b): Hardware implementation of Pedestrian Traffic Control System using an 8x8 LED Matrix using Arduino (Current state: LED Matrix in 'Stand' state and Green LED ON)

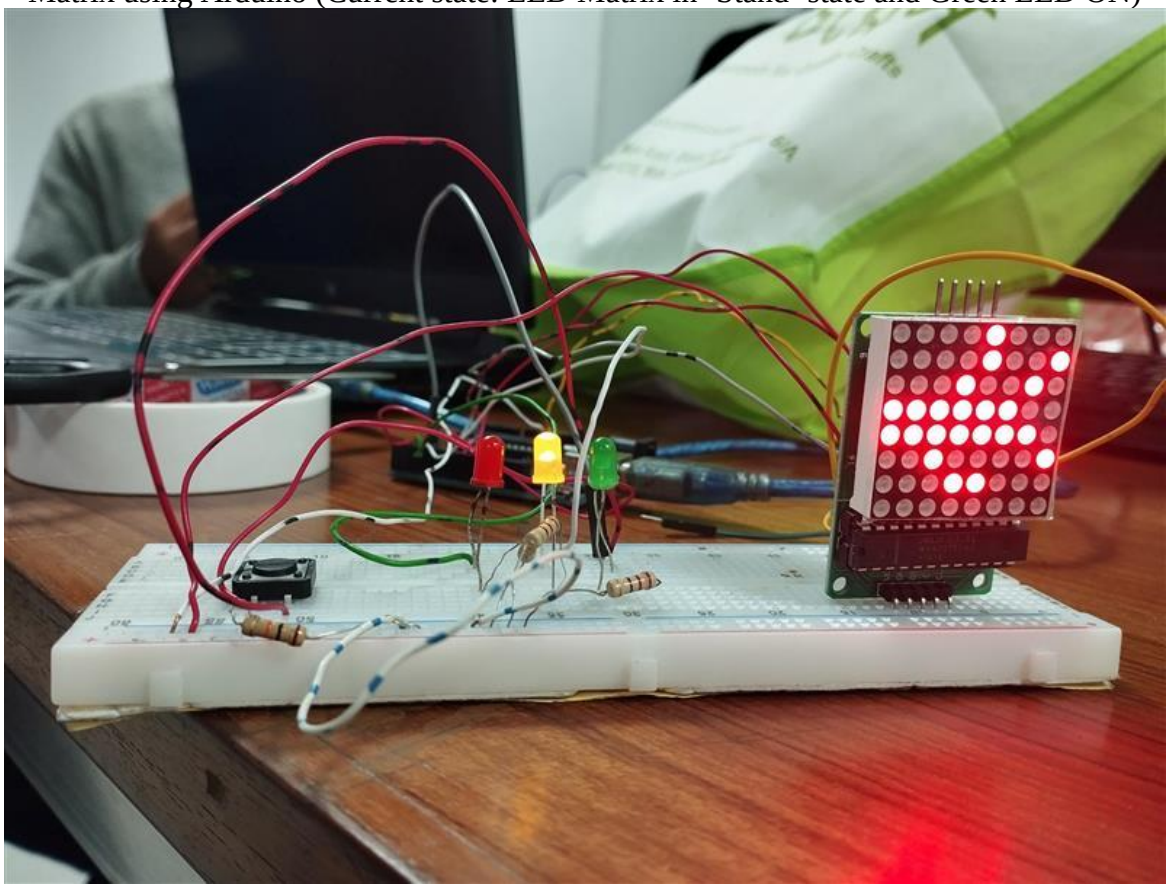

Figure 1(c): Hardware implementation of Pedestrian Traffic Control System using an 8x8 LED Matrix using Arduino (Current state: LED Matrix in 'Go' state and yellow LED Blinking)

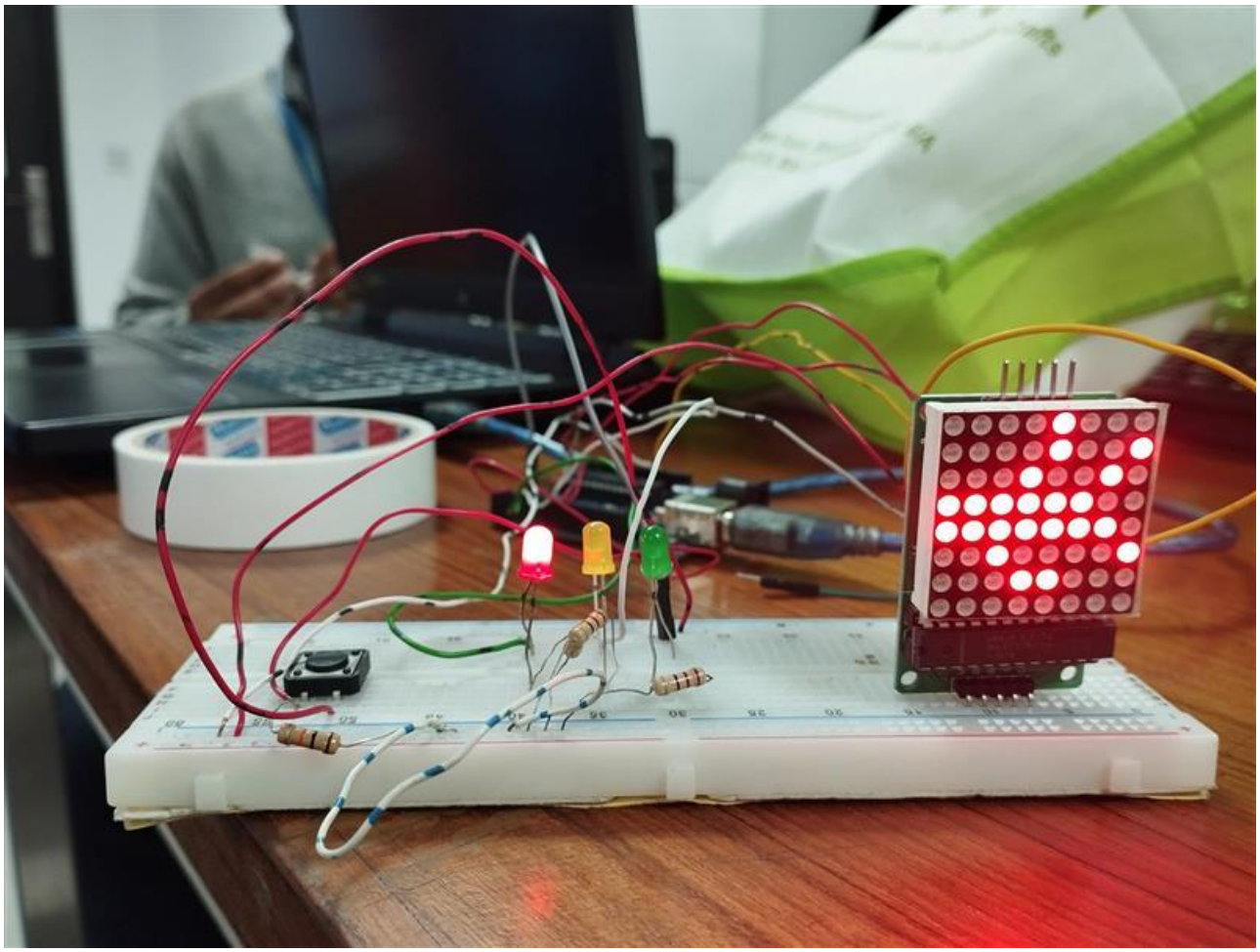

Figure 1(d): Hardware implementation of Pedestrian Traffic Control System using an 8x8 LED Matrix using Arduino (Current state: LED Matrix in 'Go' state and RED LED ON)

Simulation Result:

In Proteus, Arduino Uno, MAX7219 8x8 LED Matrix, LEDs, resistor, Pushbutton is Added. Then, using Then the circuit was put together as the circuit diagram and hardware implementation performed in lab. The pedestrian traffic control system code HEX file was inserted into the Arduino Uno Board source code. After running the simulation, the result was obtained (Figure 2(a), 2(b), 2(c), which was compared with hardware implementation for checking accuracy of both the hardware and simulation implementation. The default state LED matrix state is stand and green LED is ON. When the button is pressed, the LED matrix is in go state and yellow LED blinks 5s, then Red LED stays ON for 15s and again the yellow LED blinks for 5s. Then it reverts back to the default state.

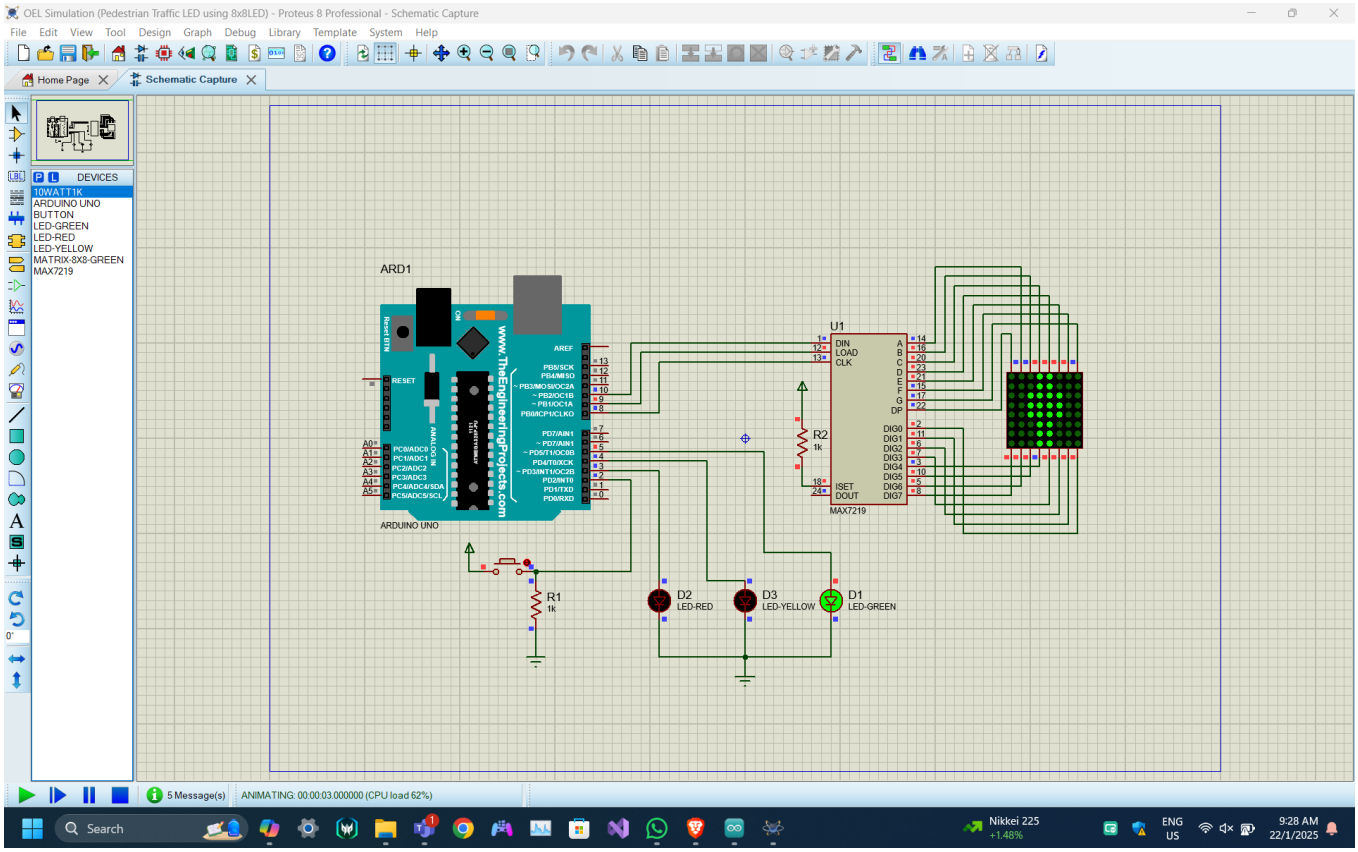

Figure 2(a): Simulation of Pedestrian Traffic Control System using an 8x8 LED Matrix using Arduino (Current state: LED Matrix in 'Stand' state and Green LED ON)

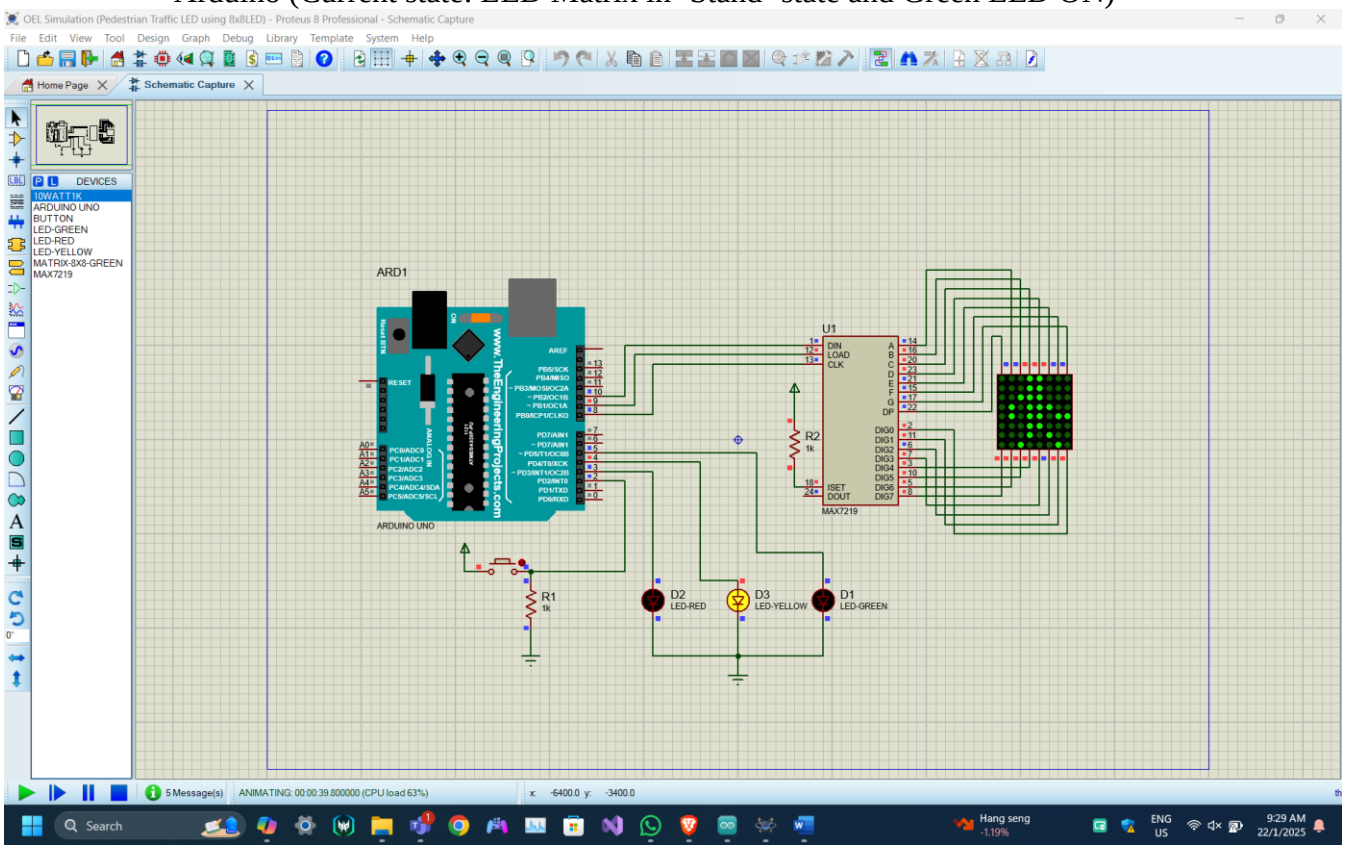

Figure 2(b): Simulation of Pedestrian Traffic Control System using an 8x8 LED Matrix using Arduino (Current state: LED Matrix in 'Go' state and Yellow LED blinking)

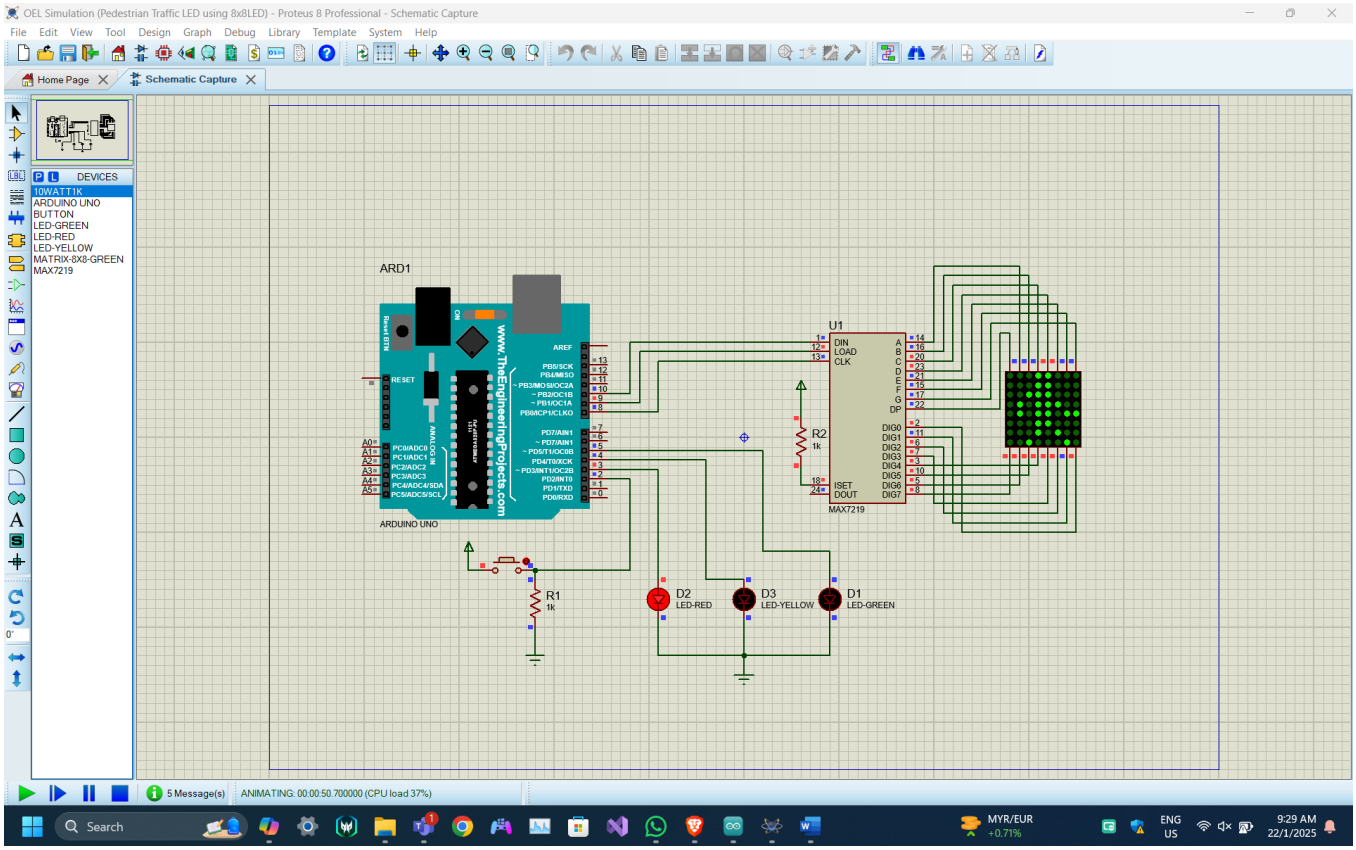

Figure 2(c): Simulation of Pedestrian Traffic Control System using an 8x8 LED Matrix using Arduino (Current state: LED Matrix in 'Go' state and Red LED ON)

Codes of the Program:

```
#include "LedControl.h"

#define CLK 8
#define CS  9
#define DIN 10
#define BUTTON_PIN 2
#define LED_PIN_3 3  //red
#define LED_PIN_4 4  // yellow
#define LED_PIN_5 5  //green

LedControl Display = LedControl(DIN, CLK, CS, 0);

byte stand[8] = {0x18,0x18,0x3C,0x3C,0x3C,0x18,0x18,0x18,};
byte go[8] = {0x18,0x18,0x38,0x5C,0x5B,0x18,0x14,0x22,};

void printByte(byte character[]) {
  for (int i = 0; i < 8; i++) {
    Display.setRow(0, i, character[i]);
  }
}

void setup() {
```

```
  pinMode(BUTTON_PIN, INPUT);
  pinMode(LED_PIN_3, OUTPUT);
  pinMode(LED_PIN_4, OUTPUT);
  pinMode(LED_PIN_5, OUTPUT);



  Display.shutdown(0, false);       // The MAX72XX is in power-saving mode on startup
  Display.setIntensity(0, 15);      // Set the brightness to maximum value
  Display.clearDisplay(0);          // and clear the display
}

void loop() {
  // Main loop does nothing, all the action happens in the interrupt

  int buttonState = digitalRead(BUTTON_PIN);  // Read the state of the button

  if (buttonState == HIGH) {
    printByte(go);

    // Turn off LED 3
    digitalWrite(LED_PIN_5, LOW);

    // Blink on LED 4 for 5 seconds
    for (int i = 0; i<5; i++)
    {
      digitalWrite(LED_PIN_4, HIGH);
      delay(500);
      digitalWrite(LED_PIN_4, LOW);
      delay(500);
    }

    // Turn on LED 5 for 20 seconds
    digitalWrite(LED_PIN_3, HIGH);
    delay(15000);
    digitalWrite(LED_PIN_3, LOW);


    for (int i = 0; i<5; i++)
    {
      digitalWrite(LED_PIN_4, HIGH);
      delay(500);
      digitalWrite(LED_PIN_4, LOW);
      delay(500);
    }

    // Turn on LED 3 again
    digitalWrite(LED_PIN_5, HIGH);

    // Change LED matrix back to smile after 30 seconds

    printByte(stand);
```

```
  }
  else {
    printByte(stand);
  digitalWrite(LED_PIN_5, HIGH);
  digitalWrite(LED_PIN_4, LOW);
  digitalWrite(LED_PIN_3, LOW);

  }

}
```

Data collection table/comparison table based on results:

Aspect	Code Description	Expected Results
Pin Definitions	Defines specific pins for LEDs, LED Matrix and Pushbutton	Physical connections between Arduino and the LED matrix are established.
Setup Function	Initializes pin modes for rows and columns as outputs, sets initial state (rows HIGH, columns LOW)	LED matrix is prepared for animation.
Designing the LED Matrix State	SelectRow function selects a specific row, Set_LED_in_Active_Row sets state of LED based on the pedestrian image.	Individual rows and LEDs are controlled according to the animation sequence.
Initializing the button	Initializing what the push button do	Button press changes the state of the LEDs and LED Matrix

Discussions:

The experiment, "Implementing Pedestrian Traffic Control using an 8x8 LED Matrix with Arduino," provides a practical introduction to traffic signal design and Arduino programming. The objectives aim to teach fundamental traffic control principles, proficiency in Arduino interface, signal timing techniques, and independent design for signaling pedestrian crossing on an 8x8 LED matrix. Traffic signals play a crucial role in ensuring pedestrian safety, and the 8x8 LED matrix serves as a versatile display to simulate traffic lights.

In this setup, each LED on the matrix represents a part of the traffic signal display. The theory covers the basics of pedestrian traffic signals, color-coding, timing, and the concept of multiplexing for efficient LED control. Participants learn about the visual representation of green (walk), red (stop), and potentially yellow (prepare to stop) signals, and how to transition between them.

The methodology takes participants through the fundamentals of traffic signal design, from initial setup to the Arduino configuration, emphasizing hands-on code manipulation for signal control. The experiment utilizes the Arduino Integrated Development Environment (IDE), an Arduino Microcontroller, and an 8x8 Dot Matrix LED Display Module. This combination ensures participants can explore dynamic signal visualizations and encourages creative autonomy in simulating pedestrian traffic control, thus fostering an engaging learning environment that merges safety engineering and microcontroller programming within a compact setup.

Course Outcome Mapping with the OEL:

CO/ CLO Number	CO/CLO Statement	K	P	A	Assessed Program Outcome Indicator	BNQF Indicator	Teaching-Learning Strategy	Assessment Strategy
1	Simulate laboratory experiments using microcontrollers, sensors, actuators switches, display devices, etc., and a suitable simulator related to the fields of electrical and electronic engineering.		P1, P4, P5		P.e.2.P4	FS.6	Practical Demonstration	OEL Report

Course Name:	Microprocessor and Embedded System	**Course Code:**	EEE 4103
Semester:	Fall 2024-25	**Section:**	J
Faculty Member:	**Prof. Dr. Engr. Muhibul Haque Bhuyan**		

OEL Title:	**Implementation of Pedestrian Traffic Control System using an 8x8 LED Matrix using Arduino.**
Group #:	01

SL	Student ID #	Name	Obtained Marks
1.	**22-46472-1**	**Ahsanul Ekram Patwary**	
2.	**22-47085-1**	**Rakibul Hassan**	
3.	**22-49373-3**	**Bijoy Ahamed**	
4.	**20-43346-1**	**Shahnewaz Ahmed**	

Assessment Materials and Marks Allocation:

COs	CO Statement	Assessment Materials	POIs	Marks
CO1	*Simulate laboratory experiments using microcontrollers, sensors, actuators switches, display devices, etc., and a suitable simulator related to the fields of electrical and electronic engineering.*	Open Ended Laboratory Report	P.e.2.P4	10

Assessment Rubrics:

COs-POIs	Excellent [9-10]	Proficient [7-8]	Good [5-6]	Acceptable [3-4]	Unacceptable [1-2]	No Response [0]	Secured Marks
CO1 P.e.2.P4	The OEL developed as a process for complex engineering problems considering microcontrollers, sensors, switches, display devices, etc. The simulation and implementation processes are clearly demonstrated combining all input patterns with several outcomes.	The OEL developed as a process for complex engineering problems considering microcontrollers, sensors, switches, display devices, etc. The simulation and implementation processes are clearly demonstrated with some outcomes and limited input patterns.	The OEL developed as a process for complex engineering problems considering microcontrollers, sensors, switches, display devices, etc. The simulation and implementation processes are not clearly demonstrated with some outcomes and input patterns.	The OEL developed as a process for complex engineering problems considering microcontrollers, sensors, switches, display devices, etc. The simulation and implementation processes are not clearly demonstrated with a few outcomes for a few patterns.	The OEL developed as a process for complex engineering problems considering microcontrollers, sensors, switches, display devices, etc. are not appropriate. The simulation and implementation processes are not demonstrated with any outcomes and not for any pattern.	No Response at all/copied from others/ identical submissions with gross errors/image file printed	
Comments					**Total marks (10)**		